Nonlinear Programming

BUS 104: Decision Analysis and Management Science

Danko Turcic

UC RIVERSIDE UNIVERSITY OF CALIFORNIA School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Today's Roadmap: A 5-Part Journey

Part 1: NLP Fundamentals & 3D Geometry

- Continuous decision variables & smooth 3D interior peaks.

Part 2: Two Leading NLP Algorithms

- **Hill-Climbing:** Fast steps along local slopes.
- **Evolutionary Search:** Population search for multi-peak hills.

Part 3: `scipy.optimize` in Python

- Step-by-step guide to solving NLPs in **Google Colab**.

Part 4: Example 1 — Retail Order Quantity (Q)

- Ordering Q units facing downward-sloping demand $P(Q) = 100 - 0.5Q$.
- Finding the peak Q^* via test steps and Python.

Part 5: Example 2 — 3-Stock Portfolio (Case Study)

- Direct foundation for your **Portfolio Optimization Case Study**.
- Choosing weights (w_1, w_2, w_3) to minimize nonlinear risk (σ_p^2).

Why We Use Python (Google Colab) for Nonlinear Programming

- **Built for Curves and Interactions:**
  - In Nonlinear Programming, formulas involve powers (Q^2), square roots ($\sqrt{x}$), and cross-products ($w_1 w_2$).
  - Writing these as clean Python functions is much clearer and less error-prone than nested grid formulas.
- **Two Industrial-Strength Solvers Built Right In:**
  - Google Colab comes pre-loaded with `scipy.optimize`—no add-ons or installations required.
  - Gives us instant access to both **Hill-Climbing (`minimize`)** and **Evolutionary Search (`differential_evolution`)**.
- **Scales Directly to Your Portfolio Case Study:**
  - The exact 10-line Python script we build today will power your upcoming **Portfolio Optimization Case Study**.

The Optimization Family Tree: LP vs. ILP vs. NLP

Key Rule: All NLP Decision Variables Are Continuous!

Just like Linear Programming (LP), **Nonlinear Programming (NLP) allows continuous fractional values** (such as $Q = 74.6$ units or portfolio weight $w_1 = 0.35$). What changes from LP to NLP is the **curved shape of the formulas**, not variable divisibility!

Feature	**Linear Prog. (LP)**	**Integer Prog. (ILP)**	**Nonlinear Prog. (NLP)**
Decision Variables	**Continuous** ($x_i \geq 0$)	Whole Numbers ($x_i \in \mathbb{Z}$)	**Continuous** ($x_i \geq 0$)
Objective / Constraints	Straight Lines ($\sum c_i x_i$)	Straight Lines ($\sum c_i x_i$)	**Smooth Curves** ($x_i^2, x_1 x_2, \sqrt{x}$)
Where Is the Peak?	Always a **Corner Point**	Grid Lattice Point	**Interior**, Boundary, or Corner
Python Library	`pulp`	`pulp` (`LpInteger`)	`scipy.optimize`

Where Nonlinearity Comes From in Business (Part A)

1. Price-Volume Interaction (Q^2)

- To sell a larger quantity Q, a company often has to lower its selling price $P(Q)$:

$$P(Q) = 100 - 0.5Q$$

- Total Revenue is Price $\times$ Quantity:

$$R(Q) = (100 - 0.5Q) \cdot Q = 100Q - 0.5Q^2$$

- The $-0.5Q^2$ term bends revenue into a **curved hill**!

2. Diminishing Returns to Ad Spend ($\sqrt{x}$)

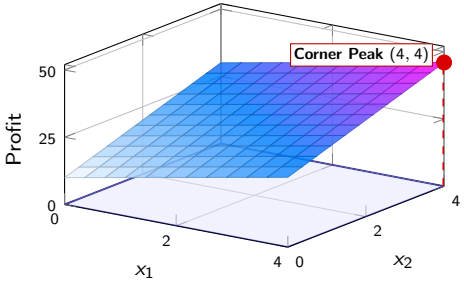
- The first $\$1,000$ spent on digital ads reaches your most eager customers.
- The 50th $\$1,000$ reaches saturated audiences and brings fewer new buyers:

$$\text{Sales}(x) = 500\sqrt{x}$$

- Because each extra dollar adds less benefit than the dollar before, the response curve flattens out.

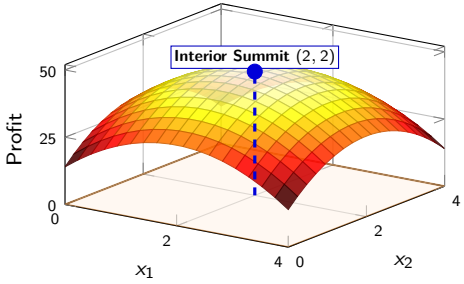
3D Geometry: Why LP Hits a Corner While NLP Peaks Inside

Linear Programming: Flat Tilted Roof



- Flat surface keeps rising until stopped by a **boundary corner**.

Nonlinear Programming: Smooth 3D Dome



- Curved surface reaches its highest point **right in the middle**!

Where Nonlinearity Comes From in Business (Part B)

3. Batch Ordering & Setup Costs (D/Q)

- Suppose annual demand is $D = 10,000$ units and you order Q units per shipment.
- The number of shipments per year is:

$$\text{Orders per Year} = \frac{D}{Q}$$

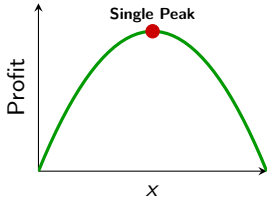
- Dividing by a decision variable Q creates a curved cost function that drops steeply at first and then flattens out.

4. Portfolio Risk & Cross-Products ($w_i^2, w_1 w_2$)

- When investing weights (w_1, w_2, w_3) across stocks, total portfolio variance includes:
  - **Squared weights:** $w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + w_3^2\sigma_3^2$
  - **Cross-product terms:** $2w_1w_2\sigma_{12}$
- These interactions capture **diversification**: how two stocks move together or offset each other!

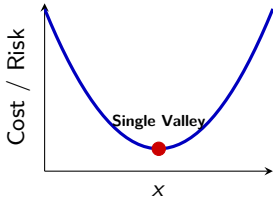
Three Shapes of Nonlinear Landscapes

1. Concave Hill (Maximize)



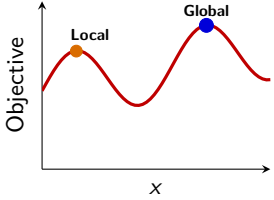
- Looks like a single smooth dome.
- Climbing uphill is **guaranteed** to find the true best peak.

2. Convex Bowl (Minimize)



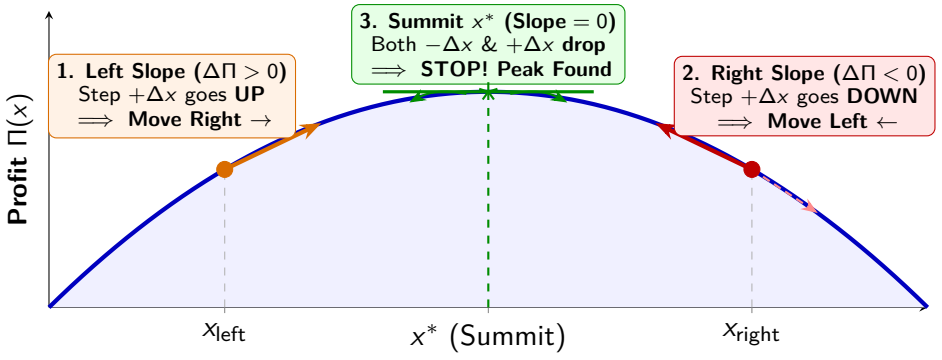
- Looks like a single smooth valley.
- Walking downhill is **guaranteed** to find the true lowest cost/risk.

3. Multi-Peak Terrain



- Contains multiple hills and valleys.
- A simple hill-climber can get trapped on a smaller **local hill**!

How Do We Know We Reached the Peak? The Test-Step Rule



Visual Rule: Nudge x left ($-\Delta x$) and right ($+\Delta x$). Climb uphill until **both directions go downhill**!

Interactive Warm-Up: Find the Peak by Testing Numbers!

5-Minute Classroom Challenge (No Calculus—Just Arithmetic!)

A subscription app earns daily profit (in \$100s) based on its monthly fee x (in dollars):

$$\Pi(x) = 40x - 2x^2 \quad \text{for } 0 \leq x \leq 20$$

1. **Why is this an NLP?** Which part of $\Pi(x) = 40x - 2x^2$ makes the graph a curve instead of a straight line?
2. **Test Three Candidate Prices:** Plug $x = 9$, $x = 10$, and $x = 11$ into $\Pi(x) = 40x - 2x^2$:
   - What is $\Pi(9)$? What is $\Pi(10)$? What is $\Pi(11)$?
3. **Identify the Summit:** Based on the Test-Step Rule, why does $x^* = 10$ have to be the profit-maximizing price?

FORMULA & PARAMETER REFERENCE: $\Pi(x) = 40x - 2x^2$ | Hint: $9^2 = 81$, $10^2 = 100$, $11^2 = 121$

Part 2: Two Leading NLP Algorithms

How Does Python Actually Find the Best Solution?

When an NLP has multiple variables and constraints, we can't just plug in numbers by hand. Python's `scipy.optimize` library gives us **two distinct search strategies**:

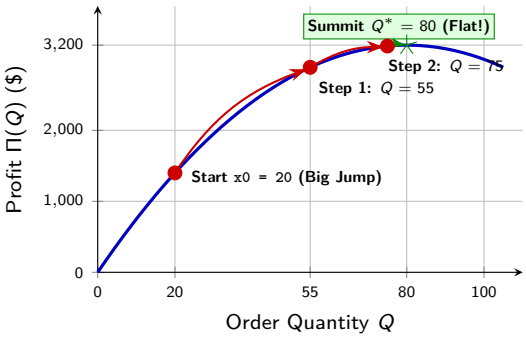
Algorithm 1: Hill-Climbing (`minimize`)

- Starts at a single initial guess `x0`.
- Tests the local slope in each direction and takes fast steps uphill (or downhill).
- **Best for:** Smooth single-peak hills or single-valley bowls.

Algorithm 2: Evolutionary Search

- Drops a whole **population** of candidate points across the entire search space.
- Combines the best solutions over generations.
- **Best for:** Bumpy multi-peak landscapes or non-smooth functions.

Algorithm 1: Hill-Climbing with Adaptive Step Sizes

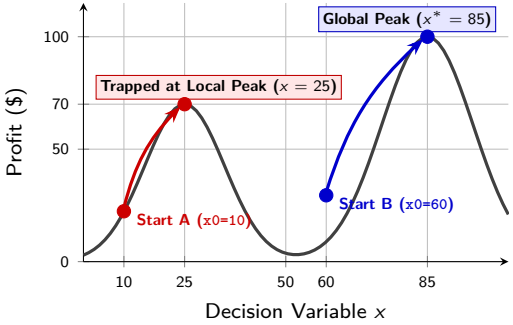


The Hiker in the Fog (`minimize`)

Starting from your initial guess x0, Python automates the Test-Step Rule with **adaptive step sizes**:

- **Steep Slope (x0 = 20):** Large improvement per unit $\Longrightarrow$ takes a **big jump** to $Q = 55$.
- **Moderate Slope ($Q = 55$):** Hill begins to curve $\Longrightarrow$ takes a **medium step** to $Q = 75$.
- **Near the Top ($Q = 75 \to 80$):** Slope is nearly flat $\Longrightarrow$ takes **tiny steps** so it doesn't overshoot $Q^* = 80$!

The Danger of Hill-Climbing: The Local Optimum Trap



Why Starting Points Matter!

- A hill-climber **refuses to walk downhill** to cross the valley at $x = 50$.
- If you start at **Start A (x0 = 10)**, it climbs to $x = 25$ (Profit = \$70) and stops—completely unaware of the taller mountain at $x^* = 85$ (Profit = \$100)!

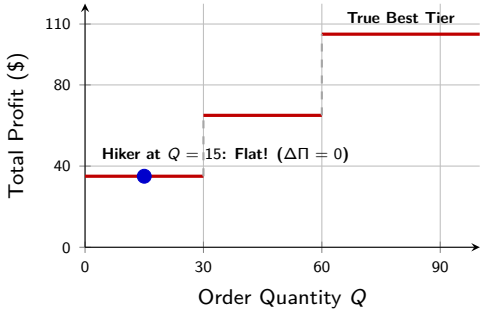
Guarding Against Local Traps: Testing Multiple Starting Points

How Do We Protect Hill-Climbing on Unfamiliar Landscapes?

If you aren't sure whether your objective function has a single peak or multiple peaks, **never rely on just one initial guess** `x0`!

- **The Multi-Start Strategy in Python:**
  1. Pick several widely separated starting guesses (for example, `x0 = [10]`, `x0 = [50]`, and `x0 = [90]`).
  2. Run `minimize(...)` from each starting guess.
  3. Compare the resulting objective values and **keep the best summit found**!
- **Good News for Our Two Main Examples Today:**
  - Both our Retail Pricing Hill (Part 4) and our 3-Stock Portfolio Risk Bowl (Part 5) have a **single peak/valley**, so every starting point converges to the exact same Global Optimum!

When Hill-Climbing Breaks Down: Steps, Cliffs, and Kinks



Why the Hiker Gets Stuck on Flat Steps

- Suppose a supplier gives sudden **step-function discounts** at $Q = 30$ and $Q = 60$, or your Python function uses `if/else`, `round()`, or `abs()`.
- At $Q = 15$, a tiny test step left or right produces **zero change** ($\Delta\Pi = 0$).
- Hill-climbing thinks it is already at the peak and quits immediately!

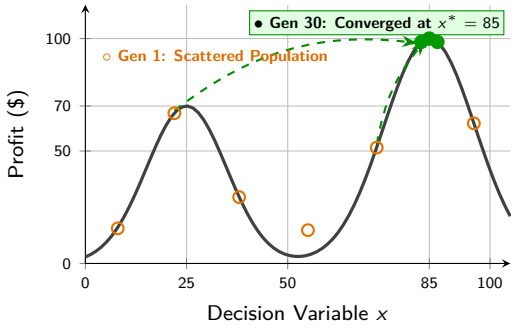
Algorithm 2: Evolutionary Search (`differential_evolution`)

No Slope Tests Needed: Inspired by Biological Evolution

Instead of walking from a single starting guess, **`differential_evolution`** searches the entire feasible region at once using a **population** of candidate solutions:

1. **Population (Generation 1):** Scatters 50–100 random candidate points across the variable bounds.
2. **Fitness Score:** Evaluates the objective formula at every point—no slope test needed!
3. **Crossover (Combining Winners):** Blends coordinates from high-performing points to create promising new candidates.
4. **Mutation (RandomJumps):** Adds random jumps so the search can leap right over valleys and cliffs!

Visual Walkthrough: Evolutionary Search Across a Bumpy Landscape



Why Evolutionary Escapes Traps

- **Generation 1 (○):** Points land on both the small hill ($x \approx 22$), the valley ($x \approx 55$), and the tall hill ($x \approx 72, 96$).
- Because points on the right hill earn higher profit (\$52–\$62), **selection and crossover** pull future generations toward $x^* = 85$.
- By **Generation 30 (•)**, the population clusters tightly around the true Global Peak!

Head-to-Head Comparison of Our Three Course Solvers

Feature	Simplex (`pulp`)	Hill-Climbing (`minimize`)	Evolutionary Search
Formula Shape	Straight Lines Only	**Smooth Curves** ($x^2, x_1x_2, \sqrt{x}$)	**Any Curve**, Step, or Cliff
How It Searches	Walks Corner to Corner	Steps Along Local Slopes	Evolves a Population of Points
Needs Starting `x0`?	No	**Yes** (Single guess `x0`)	**No** (Needs variable `bounds`)
Computation Speed	Instant	**Very Fast** (Milliseconds)	Slower (Thousands of tests)
Best Use Case	All Linear Programs	Smooth Hills & Portfolio Risk	Multi-Peak or Step Functions

Interactive Concept Check: Which Solver Should You Choose?

Quick Classroom Poll: Match Each Scenario to the Best Solver

For each business problem below, choose **(A)** `pulp` **(Linear)**, **(B)** `minimize` **(Hill-Climbing)**, or **(C)** `differential_evolution` **(Evolutionary)**:

1. **Scenario 1:** Allocating continuous weights (w_1, w_2, w_3) across 3 stocks to minimize a smooth bowl-shaped variance formula $\sigma_p^2 = \sum w_i^2 \sigma_i^2 + 2 \sum w_i w_j \sigma_{ij}$.
2. **Scenario 2:** Choosing an advertising budget $x \in [0, 500]$ when the profit curve has **four separate local peaks** and sudden step-function agency fees.
3. **Scenario 3:** Shipping cases of water from 3 warehouses to 4 stores at a constant cost of \$2.50 per case.

FORMULA & PARAMETER REFERENCE: Straight Lines → `pulp` | Smooth Single Hill/Bowl → `minimize` | Multi-Peak / Steps → `differential_evolution`

Part 3: Step-by-Step Guide to `scipy.optimize` in Python

Goal of Part 3

Before we solve business cases, let's learn the **4 building blocks** of `scipy.optimize` slowly, one piece of Python syntax at a time.

- **What We Will Cover Next:**
  1. How Python functions (`def` and `return`) represent mathematical formulas.
  2. **The Golden Rule of `scipy.optimize`:** Why all solvers minimize (and how a minus sign `-` turns minimization into maximization!).
  3. The 4-Step `scipy.optimize` pattern that works for every NLP in this course.

Meet `scipy.optimize`: Built Right Into Google Colab

- **What Is `SciPy`?**
  - `SciPy` (Scientific Python) is Python's standard library for science, engineering, and optimization.
  - Unlike `pulp`, **`scipy` is already pre-installed in Google Colab**—you never have to run `!pip install`!

- **Three Tools We Import from `scipy.optimize`:**
  1. `minimize` — Our fast Hill-Climbing solver.
  2. `differential_evolution` — Our Evolutionary Search solver.
  3. `LinearConstraint` — A clean helper for adding budget or return constraints (used in Part 5).

Python Refresher: Turning a Math Formula into a `def` Function

How Python Evaluates Formulas

In `scipy.optimize`, we give the solver a Python function (`def`) that takes a candidate decision `x` and **returns** the objective value:

```
def profit_formula(Q):
    profit = 80 * Q - 0.5 * (Q ** 2)
    return profit
```

- **Exponents in Python:** Use a double asterisk `**` for powers: Q^2 is written `Q ** 2`.
- **Indentation Matters:** The lines inside `def` are indented by 4 spaces.
- **`return`:** Hands the calculated number back to the solver so it can test whether that point is better or worse!

The Golden Rule: `scipy.optimize` Always Minimizes!

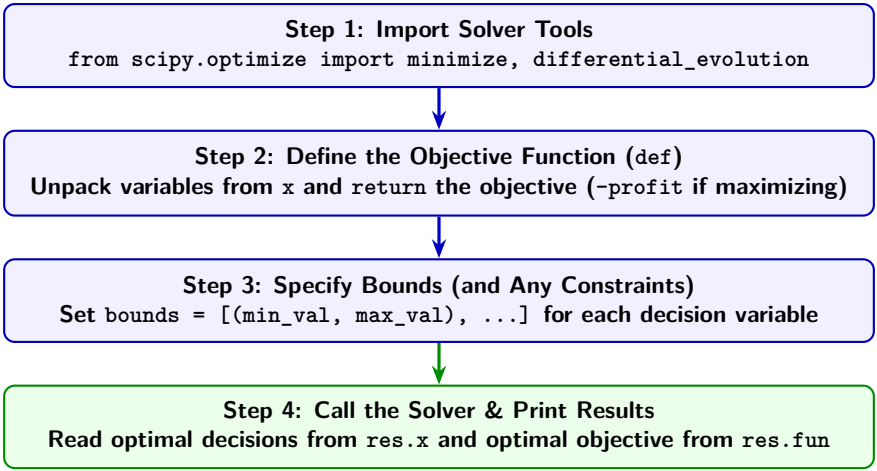
How Do We Maximize Profit If the Solver Only Minimizes?

In `scipy.optimize`, both `minimize` and `differential_evolution` always search for the **lowest valley**. To make them climb to the **highest profit peak**, we simply put a **minus sign (−)** in front of profit!

```
def neg_profit(x):
    Q = x[0]
    profit = 80 * Q - 0.5 * (Q ** 2)
    return -profit    # <-- Minus sign flips the hill upside-down!
```

- **Minimize Cost or Risk:** `return cost` (No minus sign needed!)
- **Maximize Profit or Revenue:** `return -profit` (Then flip the sign back when printing: `-res.fun`).

The 4-Step `scipy.optimize` Framework



Step 1: Importing Your Solver Tools

At the Top of Your Google Colab Notebook

We import the exact optimization tools we need from `scipy.optimize`:

```
from scipy.optimize import minimize, differential_evolution, LinearConstraint
```

- **`minimize`:** Runs our fast Hill-Climbing algorithm from a starting guess `x0`.
- **`differential_evolution`:** Runs our population-based Evolutionary Search across `bounds`.
- **`LinearConstraint`:** Lets us add linear resource or return constraints (we will use this in Part 5 for our 3-stock portfolio).

Step 2: Writing the Objective Function (def)

Why Do We Write Q = x[0] Inside the Function?

`scipy.optimize` always passes the decision variables into your function as a **list (array) called** `x`:

```
# If we have 1 decision variable (Order Quantity Q):
def neg_profit(x):
    Q = x[0]                          # Unpack the 1st variable from list x
    profit = 80 * Q - 0.5 * (Q ** 2)
    return -profit                    # Negative because we want to MAXIMIZE
```

- If you have **1 variable**, it lives at index zero: `Q = x[0]`.
- If you have **3 variables** (like 3 stock weights in Part 5), you can unpack all three in one clean line: `w1, w2, w3 = w`.

Step 3: Setting Variable Bounds (bounds)

Defining the Lowest and Highest Allowed Value for Each Variable

In Python, bounds is a **list of (min, max) pairs**—one pair for each decision variable:

```
# Example A: 1 variable Q between 0 and 200 units
bounds = [(0, 200)]

# Example B: 3 stock weights w1, w2, w3 each between 0.0 and 1.0 (0% to 100%)
bounds = [(0, 1), (0, 1), (0, 1)]
```

- Every tuple (low, high) tells Python: low $\leq$ variable $\leq$ high.
- Note: differential_evolution **requires** finite (low, high) bounds so it knows where to scatter its initial population!

Step 4: Solving & Reading the Answer (`res.x` and `res.fun`)

Running the Solver and Extracting the Numbers

Both solvers return a result object (which we name `res`) containing two key attributes:

```
# Call the Hill-Climbing solver starting from initial guess x0 = [10]
res = minimize(neg_profit, x0=[10], bounds=[(0, 200)])

print("Optimal Quantity Q* =", round(res.x[0], 2))
print("Maximum Profit ($)  =", round(-res.fun, 2))
```

- **`res.x`:** A list containing the **optimal decision variables** (so `res.x[0]` is Q^*).
- **`res.fun`:** The optimal value returned by your function (we write `-res.fun` to flip '-profit' back to positive profit!).

Part 4: Example 1 — Retail Order Quantity & Pricing (Q)

Applying Our 4-Step Python Framework to a Complete Business Problem

Let's see how a nonlinear objective naturally arises when a retailer chooses an order quantity Q facing a **downward-sloping customer demand curve**.

- **What We Will Do in Example 1:**
  1. Build the nonlinear revenue and profit formulas from basic price and cost data.
  2. Visualize why profit forms a smooth concave hill with an interior peak.
  3. Find the exact peak $Q^* = 80$ first using a simple table of test steps ($\Delta\Pi$), and then in Python using both `minimize` and `differential_evolution`.

Example 1 Scenario: Ordering Limited-Edition Campus Hoodies

The Business Decision

- The UCR Campus Store is placing a one-time order for Q **limited-edition hoodies** ($0 \leq Q \leq 200$).
- **Wholesale Cost:** Each hoodie costs the store $c = \$20$ from the supplier.
- **Price-Volume Tradeoff:** To clear a larger batch Q, the store must charge a lower retail price $P(Q)$:

$$P(Q) = 100 - 0.5Q$$

Examples of the Price Rule

- If $Q = 20$ hoodies:

$$P(20) = 100 - 0.5(20) = \$90$$

- If $Q = 80$ hoodies:

$$P(80) = 100 - 0.5(80) = \$60$$

- If $Q = 160$ hoodies:

$$P(160) = 100 - 0.5(160) = \$20$$

Building the Nonlinear Profit Formula Step by Step

Combining Revenue and Wholesale Cost

1. **Total Revenue $R(Q)$ = Price $\times$ Quantity:**

$$R(Q) = P(Q) \cdot Q = (100 - 0.5Q) \cdot Q = 100Q - 0.5Q^2$$

2. **Total Wholesale Cost $C(Q)$ = Unit Cost $\times$ Quantity:**

$$C(Q) = 20Q$$

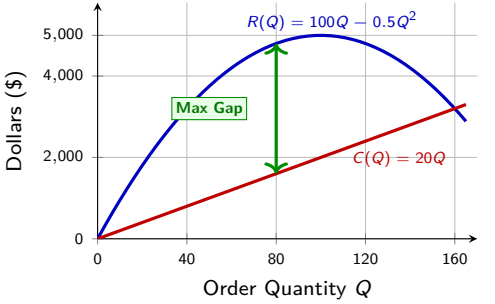
3. **Net Profit $\Pi(Q)$ = Revenue $-$ Cost:**

$$\Pi(Q) = (100Q - 0.5Q^2) - 20Q = \mathbf{80Q - 0.5Q^2}$$

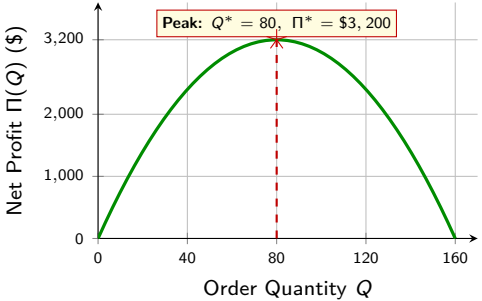
- Notice the tug-of-war: $+80Q$ pulls profit **up**, while $-0.5Q^2$ pulls profit **down** as Q grows large!

Visualizing Revenue, Cost, and the Profit Hill

1. Curved Revenue $R(Q)$ vs. Linear Cost $C(Q)$



2. Net Profit Hill $\Pi(Q) = 80Q - 0.5Q^2$



Finding the Peak Without Calculus: Testing Candidate Quantities

How Profit Changes When We Add $+1$ More Hoodie ($\Delta\Pi = \Pi(Q+1) - \Pi(Q)$)

Let's plug candidate quantities into $\Pi(Q) = 80Q - 0.5Q^2$ and check what happens when we step $+1$ unit to the right:

Quantity Q	**Price** $P(Q)$	**Profit** $\Pi(Q)$	**Next Profit** $\Pi(Q+1)$	**Step Change** $\Delta\Pi$ **(Direction)**
$Q = 20$	\$90.00	\$1,400.00	\$1,459.50	$+$\$59.50 (Climbing steeply ↗)
$Q = 50$	\$75.00	\$2,750.00	\$2,779.50	$+$\$29.50 (Still climbing ↗)
$Q = 79$	\$60.50	\$3,199.50	\$3,200.00	$+$\$0.50 (Almost at the top ↗)
$\mathbf{Q^* = 80}$	**\$60.00**	**\$3,200.00**	**\$3,199.50**	$-$**\$0.50 (Peak Reached!)**
$Q = 81$	\$59.50	\$3,199.50	\$3,198.00	$-$\$1.50 (Going downhill ↘)

- Right at $Q^* = 80$ (selling price $P^* = \$60$), stepping left to 79 or right to 81 both reduce profit to \$3,199.50!

Solving Example 1 in Python: Hill-Climbing (`minimize`)

Complete Google Colab Code Using `minimize`

Following our 4-Step Framework, we pass our function, an initial guess `x0 = [10]`, and `bounds = [(0, 200)]`:

```
from scipy.optimize import minimize

def neg_profit(x):
    Q = x[0]
    return -(80 * Q - 0.5 * (Q ** 2))

res = minimize(neg_profit, x0=[10], bounds=[(0, 200)])

print("Optimal Order Quantity Q* =", round(res.x[0], 2))
print("Maximum Net Profit ($)    =", round(-res.fun, 2))
```

Python Output

```
Optimal Order Quantity Q* = 80.0   |   Maximum Net Profit ($) = 3200.0
```

Solving Example 1 in Python: Evolutionary Search

Swapping in `differential_evolution` (Only 1 Line Changes!)

`neg_profit(x)` and `bounds` stay identical—we simply call `differential_evolution`:

```
from scipy.optimize import differential_evolution

def neg_profit(x):
    Q = x[0]
    return -(80 * Q - 0.5 * (Q ** 2))

res_evo = differential_evolution(neg_profit, bounds=[(0, 200)], seed=42)

print("Evolutionary Q*        =", round(res_evo.x[0], 2))
print("Evolutionary Profit ($)= ", round(-res_evo.fun, 2))
```

Python Output

```
Evolutionary Q* = 80.0   |   Evolutionary Profit ($) = 3200.0
```

Interactive Exercise: Supplier Cost Shock & Warehouse Cap

5-Minute Classroom Challenge (Arithmetic & Python Bounds)

Suppose the hoodie supplier raises the wholesale cost from \$20 to $\mathbf{c_{new}} = \mathbf{\$36}$ per hoodie.

- With $P(Q) = 100 - 0.5Q$, the store's new profit formula becomes:

$$\Pi_{new}(Q) = (100 - 0.5Q)Q - 36Q = \mathbf{64Q - 0.5Q^2}$$

1. **Part A (Unconstrained Peak):** Test the three candidate quantities $Q = 60$, $Q = 64$, and $Q = 70$ in $\Pi_{new}(Q) = 64Q - 0.5Q^2$. Which quantity gives the highest profit, and what is Π^*_{new}?
2. **Part B (Warehouse Cap $Q \leq 50$):** Suppose campus storage can hold **at most 50 hoodies**. How do you change `bounds = [(0, 200)]` in Python, and what will the new optimal order quantity be?

FORMULA & PARAMETER REFERENCE: $\Pi_{new}(Q) = 64Q - 0.5Q^2$ | Hint: $60^2 = 3600$, $64^2 = 4096$, $70^2 = 4900$

Part 5: Three-Stock Portfolio Optimization (Case Study Basis)

Direct Foundation for Your Upcoming Case Study!

In Nobel Prize-winning **Markowitz Portfolio Theory**, an investor allocates capital across multiple stocks to **minimize portfolio risk (variance σ_p^2)** while achieving a **target expected return**.

- **Why We Study a 3-Stock Model Today:**
  1. With 3 stocks, you can see every single term of the variance formula clearly without getting lost in huge tables.
  2. You will learn how to add **linear constraints** (`LinearConstraint`) to `scipy.optimize`.
  3. The Python code we write over the next few slides is the exact template you will use for your Case Study!

The Decision Variables: Portfolio Weights (w_1, w_2, w_3)

How We Represent an Investment Portfolio

Suppose you are investing across three stocks: **Stock 1 (Tech)**, **Stock 2 (Healthcare)**, and **Stock 3 (Utilities)**.

- **Continuous Decision Variables (w_1, w_2, w_3):**
  - w_i is the **fraction (proportion) of your total budget** invested in Stock i.
  - Example: $w_1 = 0.35,\ w_2 = 0.30,\ w_3 = 0.35$ means 35% in Tech, 30% in Healthcare, and 35% in Utilities.
- **Two Basic Rules on the Weights:**
  1. **Full Investment (Budget) Constraint:** All weights must add up to 100% (1.0):

$$w_1 + w_2 + w_3 = 1.0$$

  2. **No Short-Selling Bounds:** Each weight lies between 0 and 1: $0 \leq w_i \leq 1$.

Goal 1: Expected Portfolio Return (μ_p) Is Linear

Weighted Average of Individual Stock Returns

Each stock has an expected annual return μ_i:

- **Stock 1 (Tech):** $\mu_1 = 0.14$ (14%)
- **Stock 2 (Healthcare):** $\mu_2 = 0.10$ (10%)
- **Stock 3 (Utilities):** $\mu_3 = 0.06$ (6%)

Target Expected Return Constraint ($\mu_p \geq R_{\text{target}}$)

The portfolio's expected return is simply the weighted sum:

$$\mu_p = 0.14w_1 + 0.10w_2 + 0.06w_3$$

If our client requires an expected return of **at least** 10% **(**0.10**)**, we enforce the linear constraint:

$$0.14w_1 + 0.10w_2 + 0.06w_3 \geq 0.10$$

Goal 2: Why Portfolio Risk (Variance σ_p^2) Is Nonlinear

Why Can't We Just Average the Risks of the 3 Stocks?

Because stocks **move together or offset one another**! Total portfolio variance σ_p^2 has **two groups of nonlinear terms**:

Part A: 3 Own-Variance Terms ($w_i^2\sigma_i^2$)

$$w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + w_3^2\sigma_3^2$$

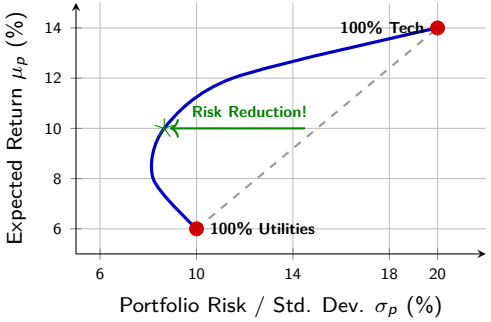
- Measures the standalone volatility of each stock (σ_i^2).
- Because weights are **squared** (w_i^2), putting 100% into one stock magnifies risk!

Part B: 3 Co-Movement Terms ($2w_iw_j\sigma_{ij}$)

$$2w_1w_2\sigma_{12} + 2w_1w_3\sigma_{13} + 2w_2w_3\sigma_{23}$$

- σ_{ij} is the **covariance** between Stock i and Stock j.
- If $\sigma_{13} < 0$ (Tech and Utilities move in opposite directions), this term **subtracts from total risk**!

Visual Intuition: How Diversification Bends the Risk Curve Inward



The Magic of Diversification

- Suppose Utilities has $\sigma_3 = 10\%$ risk and Tech has $\sigma_1 = 20\%$ risk.
- Because Tech and Utilities have **negative covariance** ($\sigma_{13} = -0.005$), mixing them cancels out part of their swings!
- The curved blue line bends **to the left** (lower risk for the exact same expected return).

Concrete 3-Stock Data & Variance-Covariance Table

Expected Annual Returns (μ_i)

- **Stock 1 (Tech):** $\mu_1 = 0.14$ (14%)
- **Stock 2 (Health):** $\mu_2 = 0.10$ (10%)
- **Stock 3 (Utilities):** $\mu_3 = 0.06$ (6%)
- **Client Target:** $\mu_p \geq 0.10$ (10%)

3×3 **Variance-Covariance Table**

Stock	Stock 1	Stock 2	Stock 3
Stock 1	$\sigma_1^2 = \mathbf{0.040}$	$\sigma_{12} = 0.003$	$\sigma_{13} = -0.005$
Stock 2	$\sigma_{12} = 0.003$	$\sigma_2^2 = \mathbf{0.020}$	$\sigma_{23} = 0.002$
Stock 3	$\sigma_{13} = -0.005$	$\sigma_{23} = 0.002$	$\sigma_3^2 = \mathbf{0.010}$

- **Diagonal (green):** Variances σ_i^2.
- **Off-diagonal (blue):** Covariances σ_{ij}.

Key Insight in the Data

Notice that $\sigma_{13} = -0.005 < 0$: Tech (Stock 1) and Utilities (Stock 3) hedge each other!

The Complete 3-Stock Mathematical NLP Formulation

Standard Mathematical Form

Decision Variables: Continuous portfolio weights w_1, w_2, w_3

$$\textbf{Minimize Risk (Variance):} \quad \sigma_p^2 = 0.040w_1^2 + 0.020w_2^2 + 0.010w_3^2 + 2(0.003)w_1w_2 + 2(-0.005)w_1w_3 + 2(0.002)w_2w_3$$

Subject to:

$$1w_1 + 1w_2 + 1w_3 = 1.0 \quad \text{(Full Investment Budget Constraint)}$$

$$0.14w_1 + 0.10w_2 + 0.06w_3 \geq 0.10 \quad \text{(Minimum 10\% Expected Return)}$$

$$0 \leq w_1, w_2, w_3 \leq 1.0 \quad \text{(No Short-Selling Bounds)}$$

- Next, let's translate this exact mathematical model into Python in **3 easy steps (Steps A, B, and C)**!

Coding 3 Stocks in Python (Step A): The Variance Function

Writing `def portfolio_variance(w):` Clearly

We unpack the three weights `w1, w2, w3 = w` and write the own-variance terms plus the co-movement terms:

```
def portfolio_variance(w):
    w1, w2, w3 = w     # Unpack the 3 stock weights from list w

    own_var = 0.040*(w1**2) + 0.020*(w2**2) + 0.010*(w3**2)
    cov_terms = 2*0.003*w1*w2 + 2*(-0.005)*w1*w3 + 2*0.002*w2*w3

    return own_var + cov_terms    # NO minus sign because we are MINIMIZING risk!
```

- **Crucial Check:** Because we want to **minimize** portfolio variance, we return `+ (own_var + cov_terms)` with **no minus sign**!

Coding 3 Stocks in Python (Step B): `LinearConstraint`

How `LinearConstraint(coeffs, min_val, max_val)` Works

In `scipy.optimize`, any linear constraint min_val $\leq c_1 w_1 + c_2 w_2 + c_3 w_3 \leq$ max_val is written with `LinearConstraint`:

```
from scipy.optimize import LinearConstraint
import numpy as np

# 1. Budget Constraint: 1*w1 + 1*w2 + 1*w3 = 1.0  (both min and max are 1.0)
budget_con = LinearConstraint([1, 1, 1], 1.0, 1.0)

# 2. Return Constraint: 0.14*w1 + 0.10*w2 + 0.06*w3 >= 0.10  (no upper limit: np.inf)
return_con = LinearConstraint([0.14, 0.10, 0.06], 0.10, np.inf)
```

- **Equality ($= 1.0$):** Set both the lower limit and upper limit to `1.0`.
- **Greater-Than-or-Equal (≥ 0.10):** Set the lower limit to `0.10` and the upper limit to `np.inf` (infinity).

Coding 3 Stocks in Python (Step C): Solving & Printing

```
bounds = [(0, 1), (0, 1), (0, 1)]
cons = [budget_con, return_con]

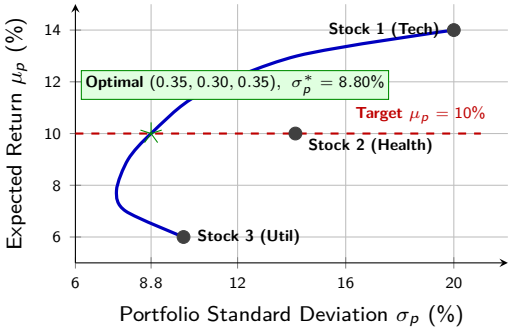
# Solve with Hill-Climbing (or swap in differential_evolution!)
res = minimize(portfolio_variance, x0=[1/3, 1/3, 1/3], bounds=bounds, constraints=cons)

print("Optimal Weights (w1, w2, w3):", [round(x, 2) for x in res.x])
print("Minimum Variance (sigma^2)  :", round(res.fun, 5))
print("Minimum Risk / Std Dev (%)  :", round((res.fun ** 0.5) * 100, 2))
```

Python Output (Both `minimize` and `differential_evolution` Agree!)

- `Optimal Weights (w1, w2, w3):  [0.35, 0.30, 0.35]` (35% Tech, 30% Health, 35% Utilities)
- `Minimum Variance (sigma^2) :  0.00775` $\Longrightarrow$ `Minimum Risk / Std Dev (%) :  8.8` (8.80%)

Visualizing the 3-Stock Markowitz Efficient Frontier



Why Diversification Wins!

- Put 100% in Stock 2 (Healthcare): you get 10% return, but your risk is $\sigma_2 = \sqrt{0.020} = \mathbf{14.14}\%$.
- Python's optimal mix $(0.35, 0.30, 0.35)$ achieves the exact same 10% return while cutting risk down to $\mathbf{8.80}\%$!

Interactive Case-Prep Exercise: Testing a 50/50 Split

5-Minute Classroom Challenge: Compare a 50/50 Split to Python's Optimum

A colleague proposes skipping Healthcare ($w_2 = 0$) and splitting the budget **50/50 between Tech and Utilities**:

$$w_{\text{cand}} = (0.50,\ 0.00,\ 0.50)$$

1. **Check Feasibility:** Does $w_{\text{cand}} = (0.50, 0.00, 0.50)$ satisfy $w_1 + w_2 + w_3 = 1.0$ and achieve the 10% target expected return ($\mu_p = 0.14w_1 + 0.10w_2 + 0.06w_3$)?
2. **Calculate Portfolio Variance (σ_p^2):** Since $w_2 = 0$, any term with w_2 disappears! Plug $w_1 = 0.50$ and $w_3 = 0.50$ into:
$$\sigma_p^2 = 0.040w_1^2 + 0.010w_3^2 + 2(-0.005)w_1 w_3$$
3. **Compare:** How does this candidate's variance compare to Python's optimal $\sigma_p^{2*} = 0.00775$?

FORMULA & PARAMETER REFERENCE: $w_1 = 0.50,\ w_3 = 0.50 \implies w_1^2 = 0.25,\ w_3^2 = 0.25,\ w_1 w_3 = 0.25$

Measuring Risk-Adjusted Return: The Sharpe Ratio

Definition: Excess Return per Unit of Risk

$$\text{Sharpe Ratio} = \frac{\mu_p - r_f}{\sigma_p}$$

where μ_p is expected portfolio return, r_f is the risk-free rate (e.g., 3% Treasury yield), and $\sigma_p = \sqrt{\sigma_p^2}$ is portfolio standard deviation (**volatility**).

Intuition & Quick Example ($r_f = 3\%$)

- **Portfolio A:** $\mu_A = 12\%$, $\sigma_A = 10\%$
  $\implies \text{Sharpe}_A = \frac{0.12 - 0.03}{0.10} = \mathbf{0.90}$
- **Portfolio B:** $\mu_B = 14\%$, $\sigma_B = 20\%$
  $\implies \text{Sharpe}_B = \frac{0.14 - 0.03}{0.20} = \mathbf{0.55}$
- **Winner:** Portfolio A delivers more return *per unit of risk*!

Python & Finsimco Case Connection

- **Optimal Point:** Maximizing the Sharpe Ratio finds the **Tangency Portfolio** on the Efficient Frontier.
- **In `scipy.optimize`:** Because `minimize()` minimizes, we minimize the **negative** Sharpe Ratio: `-(mu_p - rf) / sigma_p`.
- **In Finsimco:** Your team's percentile rank is determined by the **Sharpe Ratio** you achieve!

Case Study Launchpad: Your Portfolio Optimization Project

How Today's 3-Stock Model Connects to Your Case Study

In your upcoming **Portfolio Optimization Case Study**, you will use the exact same 4-step `scipy.optimize` workflow:

1. **Same Decision Variables:** Continuous portfolio weights $w_i \in [0, 1]$ summing to 1.0.
2. **Same Objective Function:** Minimizing portfolio variance σ_p^2 (own-variance terms + covariance terms).
3. **Same Constraints:** Full investment (`budget_con`) and minimum target return (`return_con`).
4. **What You Will Explore in the Case Study:**
   - Solving for the optimal portfolio across different target returns (R_{target}) and comparing how diversification shifts weights across assets!

Conclusion & Key Takeaways

What You Mastered Today

1. **Continuous Variables, Curved Surfaces:**
   - In NLP, decision variables remain continuous, but curved terms ($Q^2, \sqrt{x}, w_i^2, w_1 w_2$) allow the optimal peak or valley to sit **right inside the feasible region**.
2. **Two Complementary Algorithms in `scipy.optimize`:**
   - **Hill-Climbing (`minimize`):** Steps along local slopes from `x0`—super fast for smooth single peaks/bowls.
   - **Evolutionary Search (`differential_evolution`):** Evolves a population across `bounds` to escape local traps and flat steps.
3. **Ready for Practice & Your Case Study:**
   - Remember the Golden Rule (**solvers always minimize**: return `-profit` to maximize, and `+variance` to minimize risk!).